

Data Model

Design Phases

- **Initial phase** -- characterize fully the data needs of the prospective database users.
- **Second phase** -- choosing a data model
 - Applying the concepts of the chosen data model
 - Translating these requirements into a conceptual schema of the database.
 - A fully developed conceptual schema indicates the functional requirements of the enterprise.
 - Describe the kinds of operations (or transactions) that will be performed on the data.

Design Phases (Cont.)

- **Final Phase** -- Moving from an abstract data model to the implementation of the database
 - **Logical Design** -- Deciding on the database schema.
 - Database design requires that we find a "good" collection of relation schemas.
 - **Business decision** -- What attributes should we record in the database?
 - **Computer Science decision** -- What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
 - **Physical Design** -- Deciding on the physical layout of the database

Design Alternatives

- In designing a database schema, we must ensure that we avoid two major pitfalls:
 - **Redundancy**: a bad design may result in repetition of information.
 - Redundant representation of information may lead to data inconsistency among the various copies of information
 - **Incompleteness**: a bad design may make certain aspects of the enterprise difficult or impossible to model.
- Avoiding bad designs is not enough. There may be a large number of good designs from which we must choose.

Data Models

- A collection of tools for describing
 - Data
 - Data relationships
 - Data semantics
 - Data constraints
- **Relational model**
- **Entity-Relationship data model** (mainly for database design)
- Object-based data models (Object-oriented and Object-relational)
- Semi-structured data model (XML)
- Other older models:
 - Network model
 - Hierarchical model

Relational Model

- All the data is stored in various tables.
- Example of tabular data in the relational model

Columns			
ID	name	dept.name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califleri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows



Ted Codd
Turing Award 1981

(a) The *instructor* table

A Sample Relational Database

ID	name	dept.name	salary
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califleri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

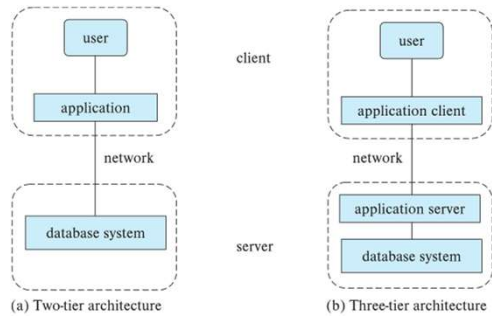
dept.name	building	budget
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Design Approaches

- **Entity Relationship Model**
 - Models an enterprise as a collection of *entities* and *relationships*
 - **Entity**: a “thing” or “object” in the enterprise that is distinguishable from other objects
 - Described by a set of *attributes*
 - **Relationship**: an association among several entities
 - Represented diagrammatically by an *entity-relationship diagram*

Two-tier and three-tier architectures



ER Model

ER model -- Database Modeling

- The ER data model was developed to facilitate database design by allowing specification of an **enterprise schema** that represents the overall logical structure of a database.
- The ER data model employs three basic concepts:
 - entity sets,
 - relationship sets,
 - attributes.
- The ER model also has an associated diagrammatic representation, the **ER diagram**, which can express the overall logical structure of a database graphically.

Entity Sets

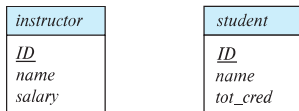
- An **entity** is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
 - Example:

$$\text{instructor} = (ID, \text{name}, \text{salary})$$

$$\text{course} = (\text{course_id}, \text{title}, \text{credits})$$
- A subset of the attributes form a **primary key** of the entity set; i.e., uniquely identifying each member of the set.

Representing Entity sets in ER Diagram

- Entity sets can be represented graphically as follows:
 - Rectangles represent entity sets.
 - Attributes listed inside entity rectangle
 - Underline indicates primary key attributes



Relationship Sets

- A **relationship** is an association among several entities

Example:

44553 (Peltier) advisor 22222 (Einstein)
 student entity relationship set instructor entity

- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

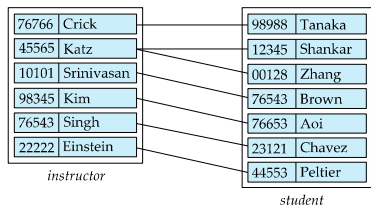
where $(e_1, e_2, \dots, e_n)$ is a relationship

- Example:

$(44553, 22222) \in \text{advisor}$

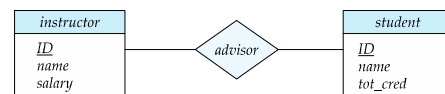
Relationship Sets (Cont.)

- Example: we define the relationship set *advisor* to denote the associations between students and the instructors who act as their advisors.
- Pictorially, we draw a line between related entities.



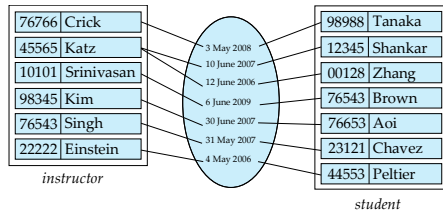
Representing Relationship Sets via ER Diagrams

- Diamonds represent relationship sets.

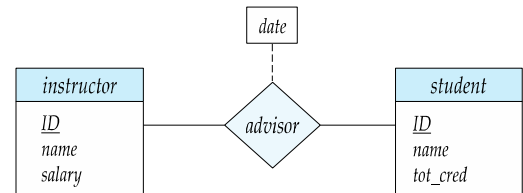


Relationship Sets (Cont.)

- An attribute can also be associated with a relationship set.
- For instance, the *advisor* relationship set between entity sets *instructor* and *student* may have the attribute *date* which tracks when the student started being associated with the advisor

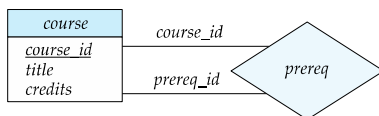


Relationship Sets with Attributes



Roles

- Entity sets of a relationship need not be distinct
 - Each occurrence of an entity set plays a “role” in the relationship
- The labels “*course_id*” and “*prereq_id*” are called **roles**.

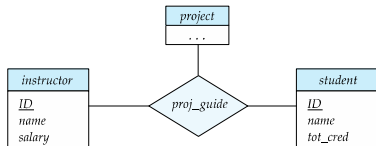


Degree of a Relationship Set

- Binary relationship**
 - involve two entity sets (or degree two).
 - most relationship sets in a database system are binary.
- Relationships between more than two entity sets are rare. Most relationships are binary.
 - Example: *students* work on research *projects* under the guidance of an *instructor*.
 - relationship *proj_guide* is a ternary relationship between *instructor*, *student*, and *project*

Non-binary Relationship Sets

- Most relationship sets are binary
- There are occasions when it is more convenient to represent relationships as non-binary.
- E-R Diagram with a Ternary Relationship

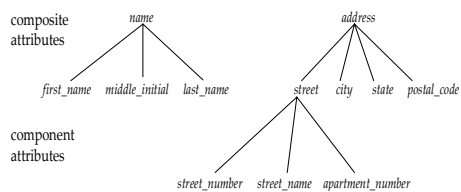


Complex Attributes

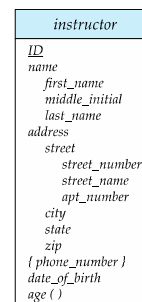
- Attribute types:
 - Simple** and **composite** attributes.
 - Single-valued** and **multivalued** attributes
 - Example: multivalued attribute: *phone_numbers*
 - Derived** attributes
 - Can be computed from other attributes
 - Example: *age*, given *date_of_birth*
- Domain** – the set of permitted values for each attribute

Composite Attributes

- Composite attributes allow us to divided attributes into subparts (other attributes).



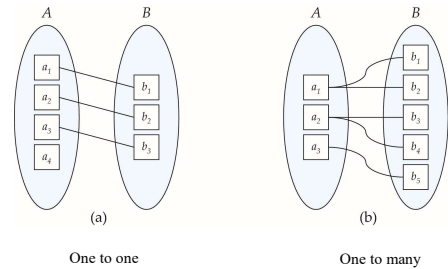
Representing Complex Attributes in ER Diagram



Mapping Cardinality Constraints

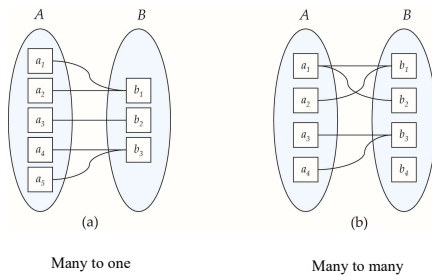
- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many

Mapping Cardinalities



Note: Some elements in *A* and *B* may not be mapped to any elements in the other set

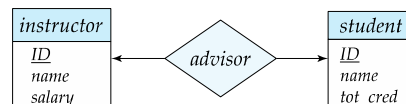
Mapping Cardinalities



Note: Some elements in *A* and *B* may not be mapped to any elements in the other set

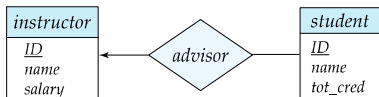
Representing Cardinality Constraints in ER Diagram

- We express cardinality constraints by drawing either a directed line ($\rightarrow$), signifying "one," or an undirected line (---), signifying "many," between the relationship set and the entity set.
- One-to-one relationship** between an *instructor* and a *student* :
 - A student is associated with at most one *instructor* via the relationship *advisor*
 - A *student* is associated with at most one *department* via *stud_dept*



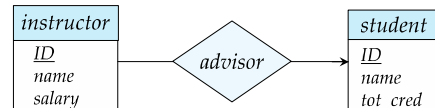
One-to-Many Relationship

- one-to-many relationship between an *instructor* and a *student*
 - an instructor is associated with several (including 0) students via *advisor*
 - a student is associated with at most one instructor via *advisor*,



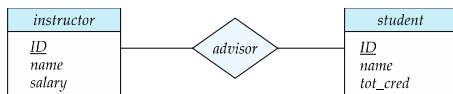
Many-to-One Relationships

- In a many-to-one relationship between an *instructor* and a *student*,
 - an instructor is associated with at most one student via *advisor*,
 - and a student is associated with several (including 0) instructors via *advisor*



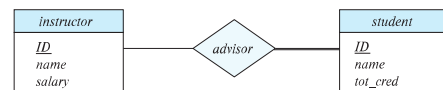
Many-to-Many Relationship

- An instructor is associated with several (possibly 0) students via *advisor*
- A student is associated with several (possibly 0) instructors via *advisor*



Total and Partial Participation

- Total participation** (indicated by double line): every entity in the entity set participates in at least one relationship in the relationship set



participation of *student* in *advisor* relation is total

- every *student* must have an associated instructor

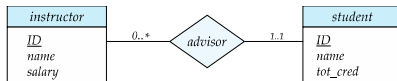
- Partial participation:** some entities may not participate in any relationship in the relationship set

- Example: participation of *instructor* in *advisor* is partial

Notation for Expressing More Complex Constraints

- A line may have an associated minimum and maximum cardinality, shown in the form $L..h$, where l is the **minimum** and h the **maximum cardinality**
 - A minimum value of 1 indicates total participation.
 - A maximum value of 1 indicates that the entity participates in at most one relationship
 - A maximum value of * indicates no limit.

Example



- Instructor can advise 0 or more students. A student must have 1 advisor; cannot have multiple advisors

Cardinality Constraints on Ternary Relationship

- We allow at most one arrow out of a **ternary (or greater degree) relationship** to indicate a cardinality constraint
- For example, an arrow from *proj_guide* to *instructor* indicates each student has **at most one guide for a project**
- If there is more than one arrow, there are two ways of defining the meaning.
 - For example, a ternary relationship R between A , B and C with arrows to B and C could mean
 - Each A entity is associated with a unique entity from B and C or
 - Each pair of entities from (A, B) is associated with a unique C entity, and each pair (A, C) is associated with a unique B
 - Each alternative has been used in different formalisms
 - To avoid confusion we outlaw more than one arrow

Primary key for Entity Sets

- By definition, individual entities are distinct.
- From database perspective, the differences among them must be expressed in terms of their attributes.
- The values of the attribute values of an entity must be such that they can uniquely identify the entity.
 - No two entities in an entity set are allowed to have exactly the same value for all attributes.**
- A **key for an entity** is a set of attributes that suffice to distinguish entities from each other

Primary Key for Relationship Sets

- To distinguish among the various relationships of a relationship set we use the individual primary keys of the entities in the relationship set.
 - Let R be a **relationship set** involving **entity sets** $E_1, E_2, \dots, E_n$
 - The primary key for R is consists of the **union of the primary keys of entity sets** $E_1, E_2, \dots, E_n$
 - If the relationship set R has **attributes** $a_1, a_2, \dots, a_m$ associated with it, then the **primary key of R** also includes the **attributes** $a_1, a_2, \dots, a_m$
- Example: relationship set "advisor".
 - The primary key consists of *instructor.ID* and *student.ID*
- The choice of the **primary key for a relationship set** depends on the mapping cardinality of the relationship set.

Choice of Primary key for Binary Relationship

- **Many-to-Many relationships:** The preceding union of the primary keys is a minimal superkey and is chosen as the primary key.
- **One-to-Many relationships:** The primary key of the “Many” side is a minimal superkey and is used as the primary key.
- **Many-to-one relationships:** The primary key of the “Many” side is a minimal superkey and is used as the primary key.
- **One-to-one relationships:** The primary key of either one of the participating entity sets forms a minimal superkey, and either one can be chosen as the primary key.

E-R Diagram (Contd..)

Problem Statement 1 :- A salesperson may manage many other salespeople. A salesperson is managed by only one salespeople. A salesperson can be an agent for many customers. A customer is managed by one salespeople. A customer can place many orders. An order can be placed by one customer. An order lists many inventory items. An inventory item may be listed on many orders. An inventory item is assembled from many parts. A part may be assembled into many inventory items. Many employees assemble an inventory item from many parts. A supplier supplies many parts. A part may be supplied by many suppliers.

- Identify the entities in the above problem statement
- Identify the relationships and cardinalities
- Identify the attributes describing entities
- Construct the E-R diagram

E-R Diagram (Contd..)

Problem Statement 2 :- Identify the entities, relationships and attributes describing those entities and construct an E-R diagram and also map them into relations with appropriate cardinalities for a hospital with a set of patients and a set of medical doctors. Each patient is characterized by its name, address, date of admission, date of discharge and whether or not person is insured. Further, a unique id is assigned to every patient and the doctor. For each doctor, their name and specialization are recorded in the system. A doctor can see zero or more patients and a patient can visit one or more doctors. A doctor recommends and performs various tests. Associated with each patient a log of the various tests and examinations conducted is maintained. For every test, test_id, test name, date_of_test and test_result are being recorded.

E-R Diagram (Contd..)

Problem Statement 3 :- Consider a shopping cart system where a customer places an Order. The system records customer id, name and address of each customer. Each order of the customer has a unique order id and order date. The order contains product items from the list of products in specific quantity and a unit price associated with it. Each product has a barcode and description associated with it.

- Identify the entities in the above problem statement
- Identify the relationships and cardinalities
- Identify the attributes describing entities
- Construct the E-R diagram

Weak Entity Sets

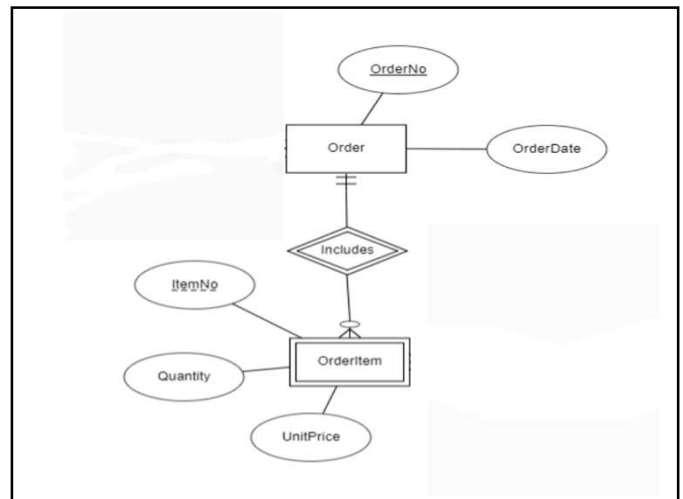
- Consider an *orderitem* entity, which is uniquely identified by a *order_id*, *quantity* and *unitprice*.
- Clearly, *orderitem* entities are related to *order* entities. Suppose we create a relationship set *includes* between entity sets *order* and *orderitem*.
- Note that the information in *includes* is redundant, since *orderitem* already has an attribute *order_id*, which identifies the *order* with which the *orderitem* is related.
- One option to deal with this redundancy is to get rid of the relationship *includes*; however, by doing so the relationship between *orderitem* and *order* becomes implicit in an attribute, which is not desirable.

Weak Entity Sets (Cont.)

- An alternative way to deal with this redundancy is to not store the attribute *order_id* in the *orderitem* entity and to only store the remaining attributes *quantity* and *unitprice*.
 - However, the entity set *orderitem* then does not have enough attributes to identify a particular *orderitem* entity uniquely
- To deal with this problem, we treat the relationship *includes* as a special relationship that provides extra information, in this case, the *order_id*, required to identify *orderitem* entities uniquely.
- A **weak entity set** is one whose existence is dependent on another entity, called its **identifying entity**
- Instead of associating a primary key with a weak entity, we use the identifying entity, along with extra attributes called **discriminator** to uniquely identify a weak entity.

Weak Entity Sets (Cont.)

- An entity set that is not a weak entity set is termed a **strong entity set**.
- Every weak entity must be associated with an identifying entity; that is, the weak entity set is said to be **existence dependent** on the identifying entity set.
- The identifying entity set is said to **own** the weak entity set that it identifies.
- The relationship associating the weak entity set with the identifying entity set is called the **identifying relationship**.
- Note that the relational schema we eventually create from the entity set *orderitem* does have the attribute *order_id*, for reasons that will become clear later, even though we have dropped the attribute *order_id* from the entity set *orderitem*.



Expressing Weak Entity Sets

- In E-R diagrams, a weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.
- Primary key for *section* – (*course_id*, *sec_id*, *semester*, *year*)



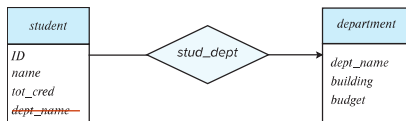
Expressing Weak Entity Sets

- In E-R diagrams, a weak entity set is depicted via a double rectangle.
- We underline the discriminator of a weak entity set with a dashed line.
- The relationship set connecting the weak entity set to the identifying strong entity set is depicted by a double diamond.
- Primary key for *section* – (*course_id*, *sec_id*, *semester*, *year*)



Redundant Attributes

- Suppose we have entity sets:
 - *student*, with attributes: *ID*, *name*, *tot_cred*, *dept_name*
 - *department*, with attributes: *dept_name*, *building*, *budget*
- We model the fact that each student has an associated department using a relationship set *stud_dept*
- The attribute *dept_name* in *student* below replicates information present in the relationship and is therefore redundant
 - and needs to be removed.



(a) Incorrect use of attribute

Reduction to Relation Schemas

Reduction to Relation Schemas

- Entity sets and relationship sets can be expressed uniformly as *relation schemas* that represent the contents of the database.
- A database which conforms to an E-R diagram can be represented by a collection of schemas.
- For each entity set and relationship set there is a unique schema that is assigned the name of the corresponding entity set or relationship set.
- Each schema has a number of columns (generally corresponding to attributes), which have unique names.

Representing Entity Sets

- A strong entity set reduces to a schema with the same attributes

student(ID, name, tot_cred)

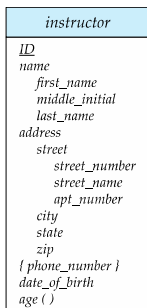
- A weak entity set becomes a table that includes a column for the primary key of the identifying strong entity set

section (*course_id*, sec_id, *sem*, *year*)

- Example



Representation of Entity Sets with Composite Attributes



- Composite attributes are flattened out by creating a separate attribute for each component attribute
 - Example: given entity set *instructor* with **composite attribute name** with **component attributes** *first_name* and *last_name* the schema corresponding to the entity set has two attributes *name_first_name* and *name_last_name*
 - Prefix omitted if there is no ambiguity (*name_first_name* could be *first_name*)
- Ignoring multivalued attributes, extended instructor schema is
 - instructor*(ID, *first_name*, *middle_initial*, *last_name*, *street_number*, *street_name*, *apt_number*, *city*, *state*, *zip_code*, *date_of_birth*)

Representation of Entity Sets with Multivalued Attributes

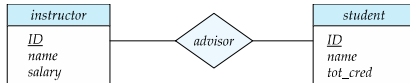
- A multivalued attribute *M* of an entity *E* is represented by a separate schema *EM*
- Schema *EM* has attributes corresponding to the primary key of *E* and an attribute corresponding to multivalued attribute *M*
- Example: Multivalued attribute *phone_number* of *instructor* is represented by a schema:

inst_phone = (*ID*, *phone_number*)
- Each value of the multivalued attribute maps to a separate tuple of the relation on schema *EM*
 - For example, an *instructor* entity with primary key 22222 and phone numbers 456-7890 and 123-4567 maps to two tuples: (22222, 456-7890) and (22222, 123-4567)

Representing Relationship Sets

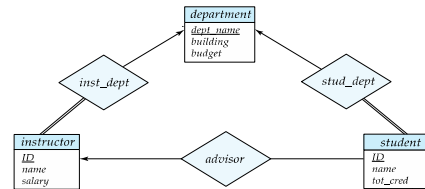
- A many-to-many relationship set is represented as a schema with attributes for the **primary keys of the two participating entity sets, and any descriptive attributes of the relationship set.**
- Example: schema for relationship set *advisor*

advisor = (*i_id, s_id*)



Redundancy of Schemas

- Many-to-one and one-to-many relationship sets** that are total on the many-side can be represented by adding an extra attribute to the “many” side, containing the primary key of the “one” side
- Example: Instead of creating a schema for relationship set *inst_dept*, add an attribute *dept_name* to the schema arising from entity set *instructor*
- Example



Redundancy of Schemas (Cont.)

- For one-to-one relationship sets, either side can be chosen to act as the “many” side
 - That is, an extra attribute can be added to either of the tables corresponding to the two entity sets
- If participation is *partial* on the “many” side, replacing a schema by an extra attribute in the schema corresponding to the “many” side could result in null values

Redundancy of Schemas (Cont.)

- The schema corresponding to a relationship set linking a weak entity set to its identifying strong entity set is redundant.
- Example: The *section* schema already contains the attributes that would appear in the *sec_course* schema



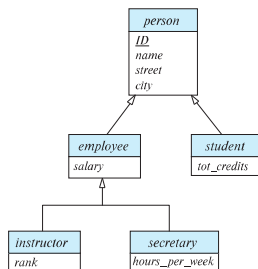
Extended E-R Features

Specialization

- Top-down design process; we designate sub-groupings within an entity set that are distinctive from other entities in the set.
- These sub-groupings become lower-level entity sets that have attributes or participate in relationships that do not apply to the higher-level entity set.
- Depicted by a *triangle* component labeled ISA (e.g., *instructor* “is a” *person*).
- **Attribute inheritance** – a lower-level entity set inherits all the attributes and relationship participation of the higher-level entity set to which it is linked.

Specialization Example

- **Overlapping** – *employee* and *student*
- **Disjoint** – *instructor* and *secretary*
- Total and partial



Representing Specialization via Schemas

- Method 1:
 - Form a schema for the higher-level entity
 - Form a schema for each lower-level entity set, include primary key of higher-level entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

- Drawback: getting information about, an *employee* requires accessing two relations, the one corresponding to the low-level schema and the one corresponding to the high-level schema

Representing Specialization as Schemas (Cont.)

- Method 2:
 - Form a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

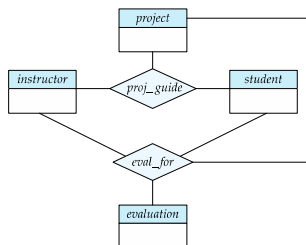
- Drawback: *name*, *street* and *city* may be stored redundantly for people who are both students and employees

Generalization

- A **bottom-up design process** – combine a number of entity sets that share the same features into a higher-level entity set.
- Specialization and generalization are simple inversions of each other; they are represented in an E-R diagram in the same way.
- The terms specialization and generalization are used interchangeably.

Aggregation

- Consider the ternary relationship *proj_guide*, which we saw earlier
- Suppose we want to record evaluations of a student by a guide on a project

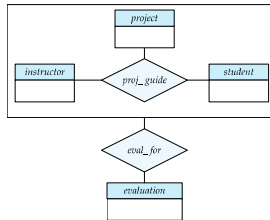


Aggregation (Cont.)

- Relationship sets *eval_for* and *proj_guide* represent overlapping information
 - Every *eval_for* relationship corresponds to a *proj_guide* relationship
 - However, some *proj_guide* relationships may not correspond to any *eval_for* relationships
 - So we can't discard the *proj_guide* relationship
- Eliminate this redundancy via *aggregation*
 - Treat relationship as an abstract entity
 - Allows relationships between relationships
 - Abstraction of relationship into new entity

Aggregation (Cont.)

- Eliminate this redundancy via *aggregation* without introducing redundancy, the following diagram represents:
 - A student is guided by a particular instructor on a particular project
 - A student, instructor, project combination may have an associated evaluation



Reduction to Relational Schemas

- To represent aggregation, create a schema containing
 - Primary key of the aggregated relationship,
 - The primary key of the associated entity set
 - Any descriptive attributes
- In our example:
 - The schema *eval_for* is:
 $eval_for(s_ID, project_id, i_ID, evaluation_id)$
 - The schema *proj_guide* is redundant.